

Reviewer Pack — Minimal Modular Form Rank and SAT Clause Set Complexity

Entry 08de5d8cea7e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 07:21:37 UTC

Minimal Modular Form Rank and SAT Clause Set Complexity

Entry ID: 08de5d8cea7e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 07:21:37 UTC

1. Conjecture

Field A (mathematical branch): Modular Form Theory **Field B** (complexity object): SAT Clause Sets

Statement:

For every k -regular graph G , the minimal rank of a modular form ($\text{mfr}(G)$) associated with its clause set is linearly correlated with the number of clauses in G , such that $\text{mfr}(G) = \Theta(|G|^{(k/2)})$.

Rationale (proposer's reasoning):

Modular forms are deeply rooted in number theory and have not been widely applied to complexity theory. The conjecture suggests a potential connection between arithmetic properties of modular forms and combinatorial structure of SAT clause sets, which could expose new insights into the nature of computational hardness.

Taxonomy category: ModularFormToSATClauseSetComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1d9110d340911da4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across at least 80% of the 30 random seeds, the ratio of the minimal rank of a modular form to the number of clauses in the graph, $\text{mfr}(G)/|G|$, is within $\pm 10\%$ of $|G|^{(k/2)}$. The conjecture is falsified if this ratio exceeds $\pm 20\%$ for any seed.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal modular form rank" AND "SAT clause sets" - "modular form theory" AND "graph clause complexity" - "linear correlation" AND "modular form rank" AND "SAT clauses"

Top relevant hits considered: - [s2:10.1007/s10601-018-9299-0] N-level Modulo-Based CNF encodings of Pseudo-Boolean constraints for MaxSAT - [s2:26ffe7a8bd90b2e805b4d94babaf8a7c4ce8e138] ARITHMETIC CLASSIFICATION OF PERFECT MODELS OF STRATIFIED PROGRAMS ON DOWNWARD CLOSURE ORDINALS OF LOGIC PROGRAMS

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_k_regular_graph(n, k):
    if (n * k) % 2 != 0:
        return None
    adj_matrix = [[0] * n for _ in range(n)]
    degree = k // 2
    nodes = list(range(n))
    random.shuffle(nodes)

    for i in range(n):
        neighbors = random.sample(nodes[:i] + nodes[i+1:], degree)
        for neighbor in neighbors:
            adj_matrix[i][neighbor] = 1
            adj_matrix[neighbor][i] = 1

    return adj_matrix

def gaussian_elimination(matrix, n):
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j

        # Swap rows
```

```

matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

# Eliminate non-pivot elements
pivot = matrix[i][i]
for j in range(n):
    if j != i:
        factor = Fraction(matrix[j][i], pivot)
        for k in range(n):
            matrix[j][k] -= factor * matrix[i][k]

return matrix

def calculate_char_poly(adj_matrix, n):
    x = 1
    char_poly = [1]
    for row in adj_matrix:
        new_poly = []
        for i in range(len(char_poly)):
            new_poly.append(x * char_poly[i] - sum(row[j] * char_poly[j] for j in range(i)))
        char_poly = new_poly

    return char_poly

def calculate_mfr(G, k):
    n = len(G)
    adj_matrix = G
    char_poly = calculate_char_poly(adj_matrix, n)

    # Calculate the minimal rank of the modular form
    mfr_G = 0
    for coeff in char_poly:
        if coeff != 0:
            mfr_G += 1

    return mfr_G

def run_trial(seed: int) -> dict:
    random.seed(seed)
    k_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in k_values:
        G = generate_k_regular_graph(n, n-1)
        if G is None:
            continue

        mfr_G = calculate_mfr(G, n-1)
        expected_value = Fraction(n**((n-1)/2))

        results.append({
            "metric_name": "mfr/G",
            "metric_value": Fraction(mfr_G, n),
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": abs(Fraction(mfr_G, n) - expected_value) <= Fraction(expected_value, n),
            "counterexample": ""
        })

```

```

    })

    return {
        "seed": seed,
        "metric_name": "mfr/G",
        "metric_value": sum(result["metric_value"] for result in results) / len(results),
        "instances_tested": len(results),
        "n_max": max(result["n_max"] for result in results),
        "conjecture_holds": all(result["conjecture_holds"] for result in results),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mfr(G)/|G| exceeds ±20%\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

Fraction(19, 240), 'instances_tested': 6, 'n_max': 40, 'conjecture_holds': False, 'counterexample': '
TRIAL: {'seed': 463, 'metric_name': 'mfr/G', 'metric_value': Fraction(19, 240), 'instances_tested': 6
TRIAL: {'seed': 503, 'metric_name': 'mfr/G', 'metric_value': Fraction(19, 240), 'instances_tested': 6
TRIAL: {'seed': 547, 'metric_name': 'mfr/G', 'metric_value': Fraction(19, 240), 'instances_tested': 6
TRIAL: {'seed': 593, 'metric_name': 'mfr/G', 'metric_value': Fraction(19, 240), 'instances_tested': 6
TRIAL: {'seed': 631, 'metric_name': 'mfr/G', 'metric_value': Fraction(19, 240), 'instances_tested': 6
TRIAL: {'seed': 677, 'metric_name': 'mfr/G', 'metric_value': Fraction(19, 240), 'instances_tested': 6
TRIAL: {'seed': 727, 'metric_name': 'mfr/G', 'metric_value': Fraction(19, 240), 'instances_tested': 6
TRIAL: {'seed': 773, 'metric_name': 'mfr/G', 'metric_value': Fraction(19, 240), 'instances_tested': 6
TRIAL: {'seed': 821, 'metric_name': 'mfr/G', 'metric_value': Fraction(19, 240), 'instances_tested': 6
TRIAL: {'seed': 877, 'metric_name': 'mfr/G', 'metric_value': Fraction(19, 240), 'instances_tested': 6
TRIAL: {'seed': 929, 'metric_name': 'mfr/G', 'metric_value': Fraction(19, 240), 'instances_tested': 6
RESULT: FALSIFIED counterexample="mfr(G)/|G| exceeds ±20%" first_failing_seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code only tests up to $n = 40$, which is too small to establish a trend that scales with n . The conjecture claims a linear correlation between $\text{mfr}(G)$ and $|G|^{(k/2)}$, but the test does not provide enough data points to confirm this relationship beyond the tested range.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The ratio of $\text{mfr}(G)$ to $|G|$ exceeded $\pm 20\%$ for at least one seed, which violates the falsification condition. | next: Increase the size of the tested graphs and retest the conjecture to confirm the trend. Investigate the cause of the deviation from the expected correlation.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15074
2	preregistration	ollama_remote	glm4:latest	0	0	10470
3	novelty	ollama_remote	glm4:latest	0	0	10751
4	novelty	ollama_remote	glm4:latest	0	0	13069
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25464
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15989
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29980
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16963
9	critic	ollama_remote	glm4:latest	0	0	17476
10	judge	ollama_remote	glm4:latest	0	0	14163

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 169400 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/08de5d8cea7e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/08de5d8cea7e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/08de5d8cea7e.tar.gz` (if generated)